AIA: 센서와 정책 가드레일의 기술 원리

Hook Sensor · File Sensor · Process Sensor · Policy Guardrails

이 문서는 AIA의 네 가지 핵심 요소를 구현 코드와 실행 흐름에 연결해 설명하는 기술 레퍼런스다. 목표는 기능 목록을 외우는 것이 아니라, 하나의 행위가 어떤 입력으로 관찰되고 어느 지점에서 정책 판정이 만들어지며 왜 특정 집행 결과를 주장할 수 있거나 없는지를 직접 추적할 수 있게 하는 것이다.

Abstract. AIA는 Codex CLI를 사용하는 Windows 11 Workspace에 보호 실행 경계를 만든다. Hook Sensor는 실행 전 Tool 요청을 AIA Session에 연결하고 정책 관문으로 보낸다. File Sensor는 Workspace에서 실제로 보이는 생성, 수정, 삭제, 이동 변화를 기록한다. Process Sensor는 AIA가 등록한 Root Process와 자식 트리를 PID와 생성 시각으로 관찰한다. Policy Engine은 Asset Type, Operation, Path Pattern 또는 Command Pattern을 결정론적으로 매칭한다. 네 요소의 결과는 하나의 추측으로 합쳐지지 않고 Raw Evidence -> Canonical Action -> Policy Evaluation + Attribution -> Enforcement -> Recovery -> Integrity -> Audit 순서로 연결된다.

이 문서에서 반드시 구분할 것

1) 관찰: 실제 시스템에서 보인 사실이다. 2) 판정: 그 사실이 정책과 맞는지의 결과다. 3) 귀속: 그 사실을 AIA Session에 어느 수준까지 연결할 수 있는지다. 4) 집행: 차단 또는 종료를 실제로 수행했는지다. 5) 복구와 무결성: 원래 상태로 되돌렸고 해시까지 검증했는지다.

이 문서의 읽는 법. 각 장은 문제 -> 구현 메커니즘 -> 실패 조건 -> 제품에서 말할 수 있는 주장 순서로 읽는다. 코드 경로는 저장소의 실제 파일을 가리키며 시연용 Fixture의 결과와 Actual Codex의 결과를 섞지 않는다.

현재 구현 기준: AIA 0.1.29 / Windows 11 / Codex CLI

AIA가 해결하려는 문제 1.

코딩 Agent는 파일 읽기, 파일 수정, 명령 실행, 자식 Process 생성 같은 시스템 행위를 수행한다. 사용자는 Agent가 반환한 설명을 읽을 수 있지만 설명만으로 실제 운영체제에서 무엇이 일어났는지 증명할 수는 없다. AIA의 설계 문제는 Agent의 내부 추론을 해석하는 것이 아니라 관찰 가능한 실행 행위를 보호된 Session에 묶고 정책과 결과를 검증 가능하게 남기는 것이다.

Agent의 자기 보고와 Security Evidence는 다르다. Agent가 파일을 만들었다고 말해도 File Sensor가 그 파일 변화를 관찰하지 못했다면 그 말은 AIA의 보안 증거가 아니다. 반대로 File Sensor가 변화를 관찰해도 어떤 Tool이 정확히 작성했는지는 별도의 귀속 문제다.

핵심 실행 사슬

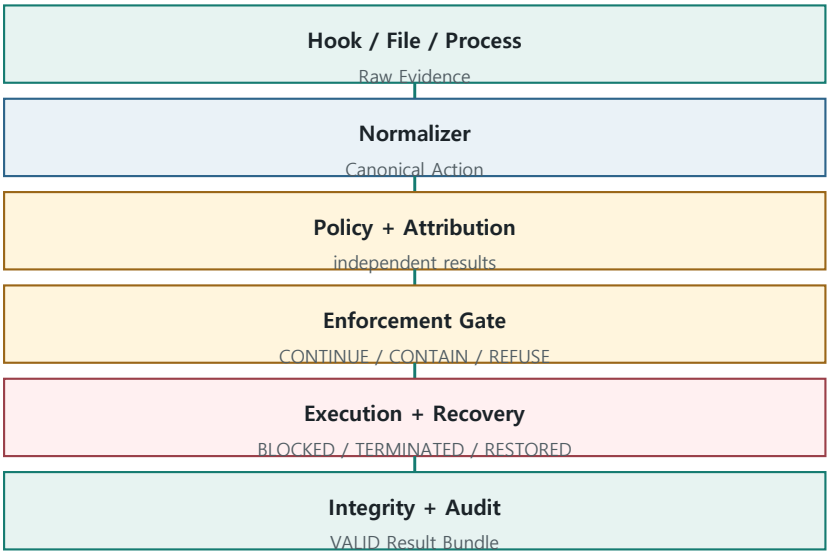


그림 1. AIA의 네 가지 핵심 요소가 결과 Bundle로 이어지는 경로

AIA는 각 단계의 의미를 덮어쓰지 않는다. Raw Evidence는 관찰 사실이고 Canonical Action은 여러 관찰을 정규화한 파생 값이며 Policy Evaluation은 정책 적합성을 판단한다. Attribution은 행위를 Agent에 연결할 수 있는 강도를 따로 계산한다. Enforcement Gate가 승인을 내린 뒤에도 실제 차단 또는 종료가 확인되기 전까지는 BLOCKED 또는 TERMINATED라고 기록하지 않는다.

구현에서 지켜지는 경계

구분	무엇을 의미하는가	잘못 섞으면 생기는 오류
Raw Evidence	Sensor가 관찰해 남긴 원시 기록	Agent 주장이나 UI 텍스트를 실제 사실로 오인
Policy Decision	관찰된 Action이 Rule에 맞는지	VIOLATION만으로 Kill 권한이 있다고 오인
Attribution	행위를 Session, Tool, Process에 연결하는 정도	시간 겹침이나 PID 하나로 EXACT 주장
Enforcement	실제 deny 또는 managed tree 종료 결과	Gate 승인만으로 BLOCKED, TERMINATED 주장
Recovery / Integrity	복구 수행과 SHA-256 비교 결과	RESTORED를 VERIFIED로 오인

`Code map: src/aia/evidence.py, normalizer.py, attribution.py, enforcement.py, result.py`

Protected Run의 공통 기반 1.1

네 센서와 정책은 아무 Workspace에 흩어져 동작하지 않는다. AIA는 먼저 Protected Run Session을 만들고 선택한 Workspace, Active Policy, Workspace Lease, Safe Checkpoint, Control Root를 연결한다. 이 공통 기반이 있어야 Evidence가 어느 실행에서 나온 것인지 설명할 수 있다.

Workspace는 사용자가 이미 만든 기존 디렉터리다. AIA가 보호 대상으로 자동 생성하지 않는다. Control Root는 Workspace 밖에 두어 Agent가 편집하는 파일과 Evidence, Lease, Result Bundle을 분리한다. 각 센서는 Evidence Store가 Session에 속하고 Control Root 아래에 있으며 Workspace 안에 있지 않은지 확인한다.

준비 단계의 의미

준비 요소	기술적 역할	실패하면
Active Policy Freeze	정책 ID와 버전을 이번 Session에 고정	실행 중 정책 변경으로 판정 의미가 변함
Workspace Lease	동일 Workspace 동시 점유 방지 및 소유 확인	두 Agent가 같은 기준선과 파일을 동시에 변경
Safe Checkpoint	Recovery와 Integrity 비교 기준선	복구는 RECOVERY_UNAVAILABLE이 될 수 있음
Control Root	Raw Evidence와 결과의 외부 보존 위치	Agent가 증거를 직접 편집할 경계가 약해짐
Sensor readiness	관찰 경로의 생존 상태 확인	보호 실행을 VERIFIED로 포장할 수 없음

> Checkpoint는 모든 집행의 전제조건이 아니다. 정책 위반을 사전 차단하는 Hook 경로와 복구에 필요한 Checkpoint는 서로 다른 책임이다. Checkpoint가 없다고 PreTool deny의 의미가 사라지는 것도 아니고 Checkpoint가 있다고 자동 종료 권한이 생기는 것도 아니다.

각 센서는 같은 질문으로 분석한다. 무엇을 입력으로 받는가? 어떤 사실만 기록하는가? 기록이 언제 만들어지는가? 어떤 정책 판단을 직접 하지 않는가? 실패할 때 왜 보수적으로 동작해야 하는가?

Hook Sensor: 실행 전에 만나는 정책 관문 2.

Hook Sensor는 Codex가 Tool을 실행하기 전에 보내는 PreToolUse 요청을 AIA가 받는 경로다. File Sensor와 Process Sensor는 이미 발생했거나 실행 중인 효과를 관찰할 수 있지만 Hook Sensor는 실행 전에 요청을 보고 정책 Gate에 전달할 기회를 가진다.

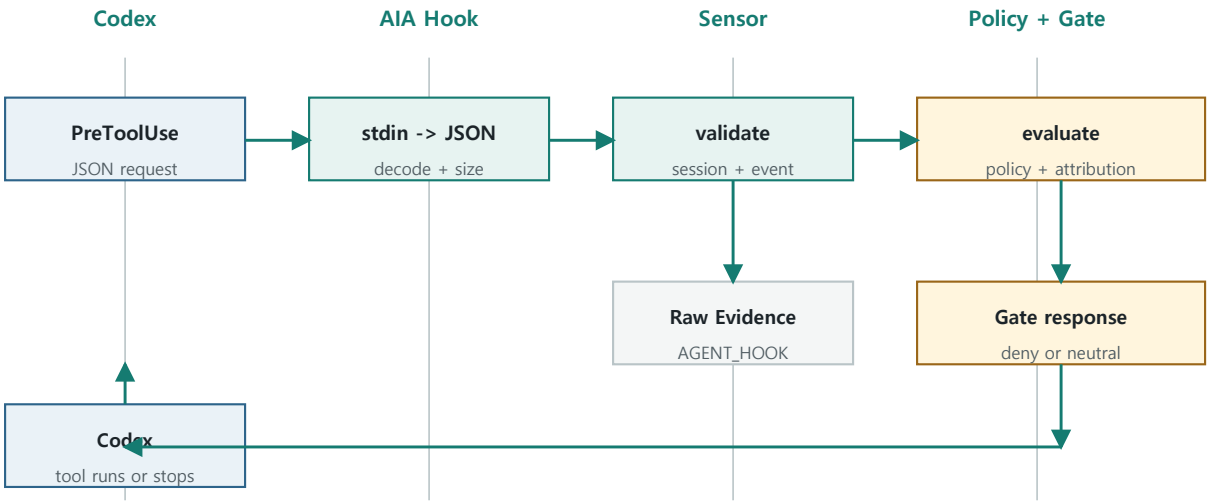


그림 2. PreToolUse 요청이 AIA Hook Sensor와 Gate를 통과하는 흐름

Hook Sensor의 역할을 한 문장으로 줄이면 다음과 같다.

Codex의 요청을 최소한의 신뢰 경계에서 검증하고 Session-bound Raw Evidence로 남긴 다음 Policy Evaluation과 Enforcement Gate의 결과를 Codex가 이해하는 PreToolUse 응답으로 변환한다.

2.1 입력 경계: JSON을 그대로 믿지 않는 이유

validate_hook_input은 stdin payload가 JSON object인지 확인하고 최대 64 KiB 크기를 넘지 않는지 검사한다. 허용된 top-level key 밖의 필드가 있으면 거부한다. 필수 필드는 aia_session_id, agent, tool_use_id, tool_name, tool_input, hook_event_name이다.

```
ValidatedHookRequest
- aia_session_id: expected Session
- agent: protected Session  agent label
- tool_use_id: Tool
- tool_name: Codex Tool
- tool_input: path, command, cwd, target
- hook_event_name:    PRE_TOOL_USE
- ingress_verified: VERIFIED        bool true
- source_metadata: adapter, channel, event_id       scalar
```

중요한 것은 많이 받는 것이 아니라 선택해서 받는 것이다. command, path, cwd, target은 이후 정책 판정에 쓰일 수 있지만 원본 Hook payload를 무제한으로 보존하지 않는다. source_metadata도 정해진 scalar key만 선택한다.

2.2 Session binding과 Trust는 다른 사실이다

확인 단계	확인하는 사실	충분한가
Hook definition	Codex 설정에 AIA command가 설치됨	설정 존재만 확인
Codex Trust	사용자가 Codex /hooks에서 PreToolUse를 검토하고 신뢰함	실행 전제조건
Live Session-bound Event	실제 보호 Session ID와 연결된 Hook Event가 관찰됨	보호 경로 연결 증명

> doctor의 READY와 hook verify의 VERIFIED를 혼동하지 않는다. READY는 사전 조건이 준비되었다는 의미다. VERIFIED는 실제 Session-bound Hook Event가 관찰되었다는 의미다.

`Code map: src/aia/sensors/hook.py, src/aia/codex_hook_runtime.py, src/aia/codex_hook_trust.py`

Hook Sensor의 판정과 실패 의미 2.3

Hook Sensor가 정상 payload를 만들었다고 곧바로 차단하는 것은 아니다. Hook Sensor는 관찰과 입력 검증을 담당하고 Policy Engine은 정책 적합성을 계산하며 Enforcement Gate는 Policy 결과 외에도 Attribution, Session, Lease, 센서 건강 상태와 실행 단계를 확인한다.

PreTool 응답은 두 가지 형태로 이해해야 한다.

Gate 상황	AIA가 Codex로 보내는 출력	실제 결과로 기록하는 값
CONTAIN + PRETOOL_BLOCK	permissionDecision = deny	Codex 실행 차단 확인 뒤 BLOCKED / SUCCEEDED
CONTINUE	빈 출력으로 native permission 처리 유지	AIA가 allow를 강제로 선언하지 않음
REFUSE	자동 집행 경로 없음 또는 검토 필요	NONE 또는 실패/검토 상태

```
AIA response when the containment route is verified:
{
  "decision": "block",
  "reason": "AIA policy violation; rule_ids=...; asset_type=...; operation=...",
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny"
  }
}

CONTINUE or REFUSE does not become an explicit allow response.
```

2.4 Gate 승인과 BLOCKED 확인 사이

PreToolEnforcementAdapter.evaluate가 Gate 결과를 deny 응답으로 바꾸는 시점과 실제 Tool이 실행되지 않았다는 확인 시점은 다르다. PreToolDecision.confirm은 confirmation.blocked가 true일 때만 BLOCKED와 SUCCEEDED를 반환한다. false이면 enforcement failure이고 None이면 block confirmation unavailable이다.

> 핵심 불변식: Policy Violation alone != Kill. Hook은 실행 전 차단 기회를 제공하지만 자동 종료는 Hook Sensor의 단독 책임이 아니다. Process termination이 필요하다면 별도의 verified post-action 경로와 Process Manager가 필요하다.

2.5 Hook 실패 시 보수적 의미

malformed JSON, 다른 Session ID, 지원하지 않는 event는 HookInputError 계열 오류가 된다. AIA endpoint가 연결된 보호 경로라면 실패를 stderr로 남기고 exit code 2를 사용한다. unbound 환경에서는 neutral output을 유지한다.

2.6 체크 질문

질문	정답
Hook이 AI의 의도를 읽는가?	아니다. 전달된 Tool 요청의 제한된 구조와 Session 연결을 검증한다.
deny 응답은 파일이 안 바뀌었다는 증거인가?	아니다. 실제 실행 부재 확인이 있어야 BLOCKED / SUCCEEDED다.
Hook 설치만으로 보호가 증명되는가?	아니다. Trust와 실제 Session-bound Event 관찰이 필요하다.
Hook이 모든 OS 행위를 관찰하는가?	아니다. 지원된 Codex Hook contract의 사전 Tool 경로다.

File Sensor: 실제 파일시스템 변화의 관찰 3.

File Sensor는 정책을 대신 결정하지 않는다. 보호 Workspace 아래에 운영체제에서 보이는 파일시스템 이벤트를 감시하고 그 이벤트를 POST_ACTION Raw Evidence로 Control Root의 Evidence Store에 append한다. 강점은 실제 경로에서 변화가 보였는가이고 약점은 누가 그 변화를 정확히 만들었는가를 단독으로 증명하지 못한다는 점이다.

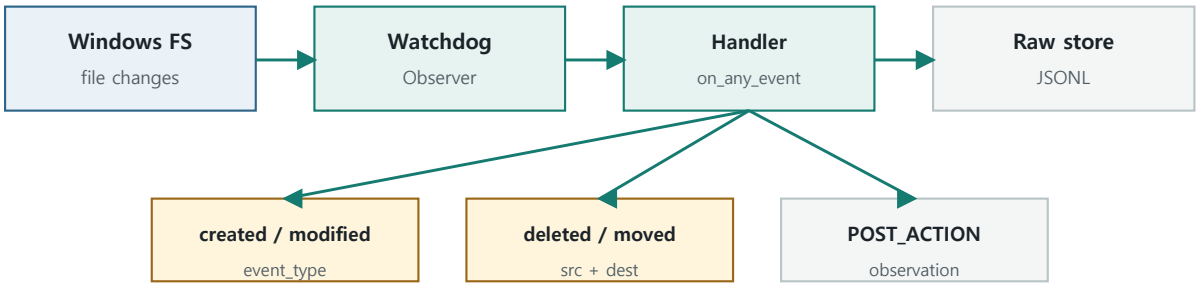


그림 3. Watchdog Observer에서 Raw Filesystem Evidence로 이어지는 흐름

3.1 Watchdog Observer와 recursive 감시

구현은 watchdog Observer를 만들고 FilesystemEventHandler를 Workspace에 recursive=True로 schedule한 뒤 start한다. Observer가 시작 직후 살아 있지 않으면 SensorStartError로 처리한다. health check에서 Observer thread가 예상치 않게 죽으면 SensorStatus.FAILED가 된다.

이 구조는 파일 읽기 API를 가로채는 방식이 아니다. watcher가 운영체제에서 전달받은 파일 변화 이벤트를 비동기적으로 처리한다. 따라서 File Sensor는 누군가 파일을 열었다보다 파일이 생성, 수정, 삭제, 이동되었다는 효과에 초점을 둔다.

3.2 지원 이벤트와 payload

event_type	기록하는 필드	기술적 의미
created	src_path, is_directory	새 파일 또는 디렉터리 생성
modified	src_path, is_directory	수정 이벤트 관찰
deleted	src_path, is_directory	삭제 이벤트 관찰
moved	src_path, dest_path, is_directory	이동 또는 이름 변경에 해당하는 경로 변화

opened, closed, access 이벤트는 지원 목록에 포함되지 않는다. 구현 주석도 이 이벤트를 READ evidence로 승격하지 않는다고 명시한다. 파일이 열렸다는 사실과 파일 내용이 읽혔다는 사실을 watcher 이벤트 하나로 같은 의미라고 주장하지 않는다.

3.3 이벤트는 어떻게 Evidence가 되는가

```
RawEvidenceRecord(
    session_id = protected Session ID,
    source = FILESYSTEM_WATCHER,
    record_type = FILESYSTEM_EVENT,
    captured_at = now_utc(),
    observation_stage = POST_ACTION,
    payload = {
        "event_type": "deleted",
        "src_path": "...",
        "dest_path": None,
        "is_directory": False
    }
)
```

handle_event는 SensorStatus가 HEALTHY가 아니면 반환한다. 지원하지 않는 event는 무시한다. src_path가 문자열이 아니면 Sensor를 FAILED로 만들고 Evidence Store append가 실패하면 EVIDENCE_STORE_FAILURE로 health를 낮춘다.

> File Sensor는 rollback을 수행하지 않는다. File Sensor는 관찰자다. Recovery Coordinator가 valid Checkpoint로 복원을 수행하고 Integrity Verifier가 expected path set과 SHA-256을 비교한다.

Code map: src/aia/sensors/filesystem.py, src/aia/evidence.py

File Sensor의 한계와 검증 3.4

파일시스템 이벤트는 관찰 시점의 운영체제 사실이지만 특정 Agent Tool에 곧바로 귀속하는 완전한 provenance는 아니다. 같은 Workspace를 여러 Process가 쓸 수 있고 이벤트 전달은 실행 요청과 시간적으로 분리될 수 있다. AIA는 이 불확실성을 Attribution 단계에서 별도로 다룬다.

관찰 가능한 것	관찰만으로 말할 수 없는 것
Session Workspace에서 delete event가 보였다	특정 Tool이 정확히 그 delete를 수행했다고 단정
move의 src_path와 dest_path가 기록되었다	이동 전후 파일 내용까지 자동으로 증명
Checkpoint와 현재 상태의 차이가 있다	변경이 자동으로 되돌아갔다고 단정
Recovery가 RESTORED를 반환했다	Integrity가 VERIFIED라고 단정

3.5 파일 삭제를 끝까지 설명하는 방법

순서	질문	근거
1	정책상 DELETE가 위반인가?	Canonical Action + Policy Rule
2	파일이 실제로 사라졌는가?	Filesystem DELETE Raw Evidence
3	기준 상태로 복원되었는가?	Recovery RESTORED + Integrity VERIFIED

세 질문을 한 줄의 AIA가 파일을 복구했다로 압축하면 기술적으로 부족하다. 복구가 수행되었지만 해시가 다르거나 파일 변화는 보였지만 기준선이 없어 RECOVERY_UNAVAILABLE인 경우를 표현하려면 결과 축을 분리해야 한다.

3.6 체크 질문

질문	정답
File Sensor가 파일 내용을 전부 저장하는가?	아니다. bounded filesystem event payload를 저장한다.
파일이 열렸다는 이벤트가 READ 증거인가?	현재 구현에서는 아니다.
File Sensor가 정책 위반을 직접 차단하는가?	아니다. 관찰을 남기고 정책과 집행에 연결한다.
File Sensor가 writer를 정확히 식별하는가?	단독으로는 아니다. 별도 Attribution이 필요하다.

Process Sensor: 관리 대상 Process Tree의 관찰 4.

Process Sensor는 AIA가 명시적으로 등록한 Root Process에서 시작해 자식 Process Tree를 관찰한다. Windows에서 PID는 숫자일 뿐이고 재사용될 수 있기 때문에 ProcessIdentity는 PID와 create_time을 함께 사용한다. 두 값이 맞지 않으면 같은 Process로 취급하지 않는다.

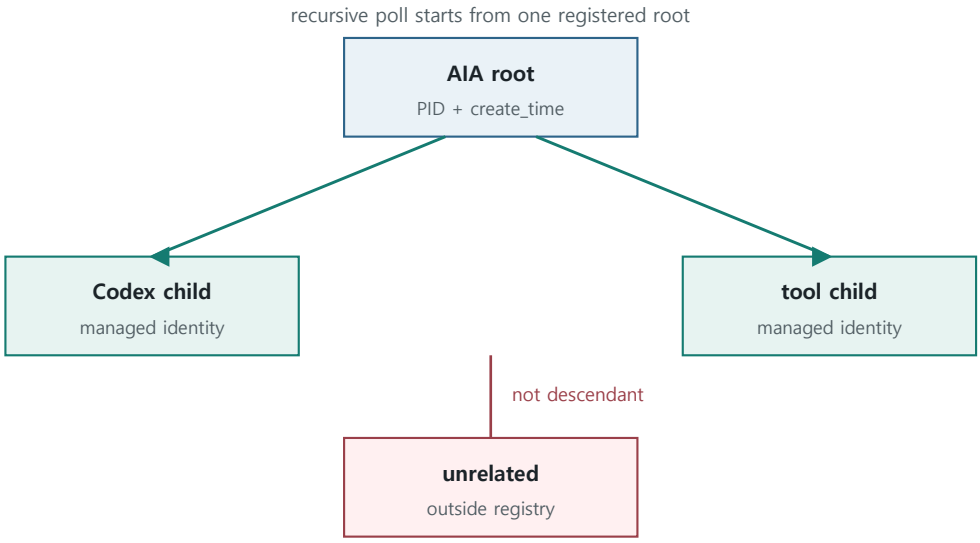


그림 4. 명시적으로 등록된 Root와 unrelated Process의 경계

4.1 psutil poller의 구조

ProcessRuntimeSensor는 기본 poll_interval 0.25초로 daemon thread를 실행한다. 시작 시 Root identity가 LIVE_MATCH인지 확인하고 즉시 한 번 tree를 수집한다. 이후 poll loop가 현재 Root와 자식들을 재귀적으로 수집한다. 처음 본 ProcessIdentity만 PROCESS_RUNTIME Raw Evidence로 append한다.

```
Root identity = (pid, create_time)
poll:
    verify root identity
    current = collect(root)
    for identity in current:
        if identity not in observed:
            append observation(identity, ppid, role, metadata)
    for previous identity missing from current:
        append exit observation
    observed.update(current)
```

4.2 ProcessObservation의 정보

필드	왜 필요한가
pid	현재 운영체제 Process handle을 찾는 기본 식별자
create_time	PID가 재사용되었는지 구분하는 생명주기 정보
ppid	부모와 자식 관계를 검증하는 관찰 값
process_role	ROOT 또는 DESCENDANT
root_identity	어느 AIA 관리 트리 소속인지 연결
name / executable	관찰된 Process의 bounded metadata
raw_command	capture_command_line이 켜진 경우에만 선택적 보존
metadata_status	메타데이터가 COMPLETE인지 제한되었는지

child enumeration과 ppid 읽기 사이에 Windows tree가 바뀔 수 있는 race도 고려한다. parent relation이 안정적으로 검증되지 않으면 그 descendant만 이번 poll에서 생략하고 나중에 다시 관찰한다.

> Process Sensor의 핵심 주장. 이 Process가 관리 Root의 현재 또는 이전 descendant였는지를 identity와 parent relation으로 관찰할 수 있다. 이 Process가 특정 파일을 썼다는 주장은 Process Sensor 단독으로 만들지 않는다.

`Code map: src/aia/sensors/process.py, src/aia/process_identity.py`

Process identity와 안전한 종료 4.3

관찰과 종료는 다른 책임이다. Process Sensor는 PROCESS_RUNTIME evidence를 만들고 Process Manager는 Enforcement Gate가 승인한 managed process identity에 대해서만 실제 termination을 수행한다. 센서가 위험해 보이는 명령행을 관찰했다고 자동으로 kill하지 않는다.

상태	뜻	보안 해석
LIVE_MATCH	PID와 현재 create_time이 모두 일치	현재 handle을 같은 Process로 취급 가능
PROCESS_GONE	대상 Process가 사라짐	이미 종료되었거나 관찰 창에서 사라짐
PID_REUSED	PID는 같지만 create_time이 다름	예전 Process를 종료하면 안 됨
OWNER_UNVERIFIABLE	권한 또는 OS 오류로 확인 불가	보수적으로 종료하지 않음

Process Manager는 Root 등록으로 시작해 managed registry에 ProcessIdentity를 보존한다. 종료 전 identity를 검사하고 종료 후에도 대상이 사라졌는지 확인한다. 등록되지 않은 unrelated Process는 registry에 없으므로 정상 managed tree termination의 대상이 아니다.

잘못된 단순화	정확한 설명
PID 1234를 kill했다	AIA가 등록한 (PID, create_time) identity를 확인한 managed Process Tree를 종료 시도했다.
자식이 있으니 모두 죽였다	Root identity에서 확인된 descendant를 수집하고 검증된 managed identity에 대해 종료 결과를 기록했다.
Process Sensor가 파일 삭제를 잡았다	Process lifecycle과 파일 변화는 별도 evidence chain이다.

4.3 Process Sensor와 Hook의 결합

Hook이 사전 차단에 실패했거나 PostToolUse에서 실제 실행이 확인된 특수 경로에서는 Process evidence가 보상 집행의 추가 근거가 될 수 있다. 그러나 그 경로도 Policy, Attribution, Session, Lease, Sensor health, Process identity를 다시 확인해야 한다. Process Sensor가 단독으로 Gate를 열지 않는 이유는 관찰과 행위 귀속을 같은 것으로 만들지 않기 위해서다.

4.4 체크 질문

질문	정답
PID만 기록하면 충분한가?	아니다. create_time을 함께 비교해야 PID reuse를 막는다.
모든 Windows Process를 감시하는가?	아니다. AIA가 명시적으로 등록한 managed tree다.
Process Sensor가 곧 Kill 기능인가?	아니다. 실제 종료는 Gate 이후 Process Manager다.
Process exit가 파일 writer 증거인가?	아니다. Process exit와 파일 변화는 별도 evidence다.

정책 설정: 가드레일은 어떻게 동작하는가 5.

정책은 AI가 나쁜 의도를 가졌는가를 판단하는 모듈이 아니다. 정책은 관찰된 행위에 대해 어떤 Asset Type과 Operation을 어떤 Pattern으로 허용, 기록 또는 금지할지 정의하는 결정 규칙이다. Policy Engine은 Attribution이나 Process termination을 수행하지 않는 순수한 결정 계층이다.

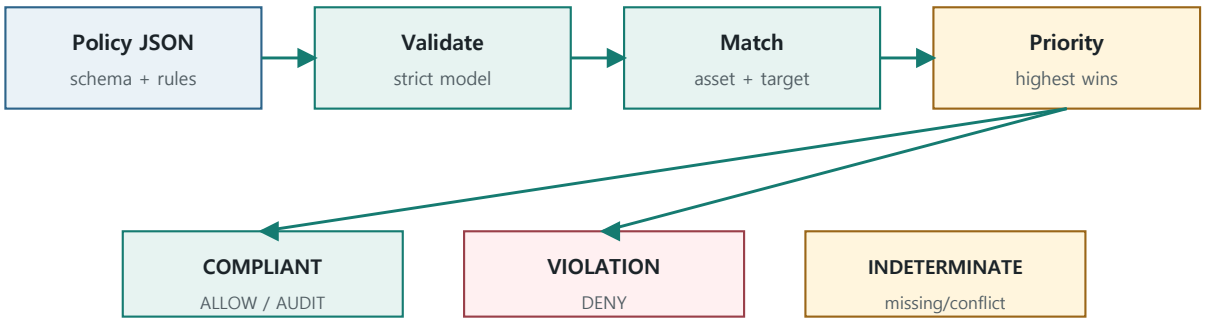


그림 5. 정책 문서가 Strict Validation과 우선순위 판정을 거치는 흐름

5.1 정책의 구성

구성	현재 구현의 값	질문
Asset Type	SECRET, DATABASE_RELATED, CRITICAL_CODE, DEPENDENCY, INFRASTRUCTURE, REPOSITORY, WORKSPACE	보호 대상 자산의 종류는?
Operation	READ, CREATE, MODIFY, DELETE, MOVE, RENAME, EXECUTE	어떤 행위를 하는가?
Target Pattern	path_patterns 또는 command_patterns 중 정확히 하나	어떤 경로 또는 명령에 적용?
Effect	ALLOW, AUDIT, DENY	Rule이 맞으면 어떤 효과?
Priority	정수	여러 Rule이 맞을 때 무엇을 우선?

5.2 File Rule과 Command Rule을 섞지 않는 이유

PolicyRule은 path_patterns와 command_patterns를 둘 다 비워 두거나 둘 다 채우는 것을 허용하지 않는다. 둘 중 정확히 하나만 있어야 한다. command_patterns가 있으면 EXECUTE effect만 허용한다. path_patterns가 있으면 파일 Operation만 허용한다.

```
# file rule
asset_type: SECRET
path_patterns: [".env", "**/.env"]
operation_effects:
  READ: DENY
  MODIFY: DENY

# command rule
asset_type: WORKSPACE
```

```
command_patterns: ["Remove-Item *", "taskkill /f *"]
operation_effects:
  EXECUTE: DENY
```

5.3 Pattern matching의 실제 의미

Path는 normalize_path를 거쳐 fnmatchcase 기반으로 비교한다. Command는 whitespace를 정리하고 casefold한 뒤 wildcard가 있으면 fnmatchcase로 비교한다. wildcard가 없으면 명령 전체가 같거나 pattern 뒤에 공백이 이어지는 command prefix로 취급한다. 이것은 lexical matching이며 문자열 일치만으로 script 내부 파일 효과를 증명하지 않는다.

5.4 Match, priority, conflict, default

단계	엔진의 동작
입력 보완	File Operation에는 path evidence, EXECUTE에는 raw command evidence가 필요
필수 evidence 부족	COMPLIANT가 아니라 INDETERMINATE
Rule match	Asset Type, Operation, path 또는 command pattern이 모두 맞는 Rule 수집
Priority	최고 priority Rule만 다음 판단에 사용
동일 priority conflict	효과가 다르면 POLICY_CONFLICT -> INDETERMINATE
No match	정책의 default_effect 사용
결과	DENY이면 VIOLATION, ALLOW 또는 AUDIT이면 COMPLIANT

> default_effect도 정책의 일부다. Rule이 없다고 자동으로 위험하거나 안전한 것이 아니다. 정책의 default_effect에 따라 결과가 달라지며 필수 target evidence가 없으면 default로 내려가기 전에 INDETERMINATE가 된다.

`Code map: src/aia/policy.py, src/aia/runtime_policy.py`

정책 가드레일을 직접 계산해 보기 5.5

예제는 비밀정보인 .env를 읽으려는 요청이다.

```
Subject
  asset_type = SECRET
  operation  = READ
  path       = C:\workspace\project\.env

Candidate rule
  R-SECRET-READ
    priority = 100
    asset_type = SECRET
    path_patterns = ["**/.env"]
    READ = DENY

Result
  highest priority = 100
  effect = DENY
  policy_decision = VIOLATION
  matched_rule_ids = ["R-SECRET-READ"]
```

AIA가 자동으로 프로세스를 죽인다고 결론 내리지 않는 것이 핵심이다. Policy Engine의 결과는 VIOLATION이다. 그 다음에는 Tool 요청의 Session binding, Hook ingress 상태, Attribution grade, Enforcement Gate의 route가 별도로 확인되어야 한다. 실행 전 PRETOOL_BLOCK route가 승인되고 Codex의 실제 deny가 확인되면 BLOCKED / SUCCEEDED를 기록한다.

5.5.1 INDETERMINATE를 만나는 예

입력	결과	왜
READ + path 없음	INDETERMINATE	파일 Operation에는 path evidence가 필수
EXECUTE + raw command 없음	INDETERMINATE	명령 Pattern을 비교할 수 없음
같은 priority의 ALLOW와 DENY	INDETERMINATE	충돌을 임의로 선택하지 않음
Rule 없음 + default AUDIT	COMPLIANT + audit	정책상 기록 대상으로 해석
Rule 없음 + default DENY	VIOLATION	기본 금지 정책

5.5.2 asset_type을 누가 결정하는가

runtime caller가 Asset Type을 임의로 붙이지 않는다. Policy Engine의 evaluate_target은 operation과 path 또는 raw command를 받아 매칭 Rule에서 asset type을 도출할 수 있다. 자산 분류도 정책 문서와 Rule matching의 결과로 추적되어야 한다.

5.6 체크 질문

질문	정답
정책이 Agent 의도를 판단하는가?	아니다. 관찰된 Action의 operation과 target을 Rule에 비교한다.
Rule이 맞으면 바로 kill하는가?	아니다. Policy Decision은 Gate 입력 중 하나다.
경로 Rule에 command pattern을 같이 넣어도 되는가?	안 된다. 한 Rule은 path 또는 command 중 하나만 가져야 한다.
evidence가 없으면 default ALLOW인가?	아니다. 필수 evidence 부족은 INDETERMINATE다.

네 요소를 하나의 사건으로 연결하기 6.

예시는 Agent가 .env를 읽으려는 경우다. Hook, File, Process, Policy는 각자 다른 시간과 책임을 가진다.

6.1 사건의 시간순서

시점	관찰 또는 판정	담당
t0	Codex가 READ .env를 요청	Hook Sensor
t1	Session ID, Tool ID, tool name, input 검증	Hook validator
t2	SECRET + READ + path pattern match	Policy Engine
t3	VIOLATION과 Session binding, route 조건 결합	Attribution + Gate
t4	Codex에 PreToolUse deny 반환	PreTool Adapter
t5	실제 Tool 실행 부재 확인	confirmation path
t6	BLOCKED / SUCCEEDED와 evidence references 기록	Finalization

이 경우 File Sensor가 아무 이벤트도 기록하지 않는 것이 정상일 수 있다. 실행 전에 차단되었다면 .env read로 인한 File Event가 없기 때문이다. 센서 성공률은 기대되는 관찰 단계와 scenario ground truth를 함께 정의해야 한다.

6.2 중요 파일 삭제 사건

파일 삭제는 Hook의 사전 차단, File의 사후 변화, Process의 실행 생명주기, Policy의 가드레일이 서로 다른 관점으로 보게 된다. 삭제가 PreTool에서 차단되면 File DELETE가 없는 것이 정상이다. 실제 효과가 생기면 File Sensor가 DELETE를 관찰하고 종료 및 복구가 별도로 진행된다.

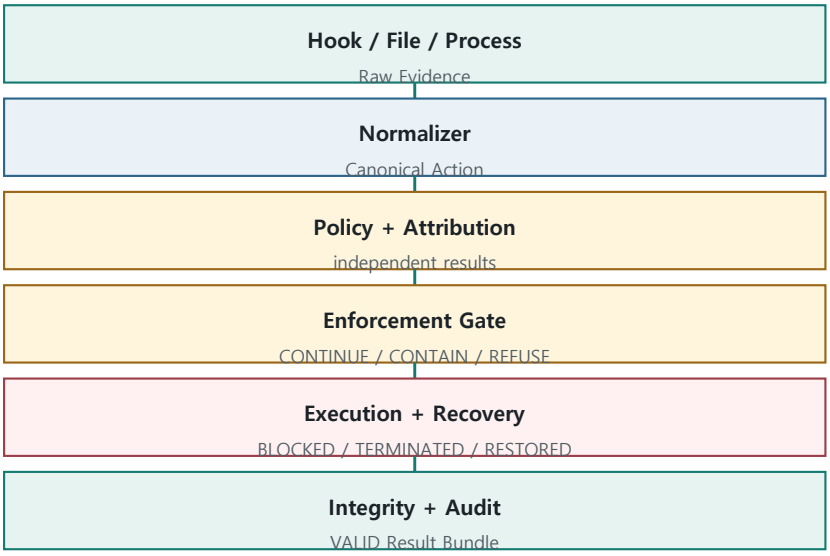


그림 6. 센서의 관찰과 정책 및 집행 결과를 분리해서 연결하는 모델

6.3 결과를 말하는 정확한 문장

부정확한 문장	정확한 문장
AIA가 위험한 AI를 알아보고 죽였다.	AIA는 관찰된 Action을 정책으로 VIOLATION 판정했고 충분한 귀속과 identity 확인 뒤 Gate를 통과한 managed Process에 종료를 수행했다.
Hook이 파일 삭제를 막았다.	Hook의 PreTool 요청에서 삭제 Action을 사전 차단했고 실제 Tool 실행 부재가 확인된 경우 BLOCKED / SUCCEEDED를 기록했다.
File Sensor가 파일을 복구했다.	File Sensor가 DELETE를 관찰했고 Recovery가 Checkpoint로 RESTORED를 수행한 뒤 Integrity가 VERIFIED를 확인했다.
Process Sensor가 파일 작성자를 증명했다.	Process Sensor가 managed Process identity와 lifecycle을 관찰했다. 파일 writer 귀속은 별도 evidence 조합이 필요하다.

시연과 정량 평가를 읽는 법 6.4

결정론적 Fixture는 흐름과 결과 화면을 안정적으로 보여주기 위한 데이터다. Actual Codex Hook 검증은 실제 Codex에 Hook을 연결해 Session-bound Event를 확인하는 별도의 증명이다.

케이스	보여주는 기술 원리	주장할 수 있는 결과
process-kill	managed Root/descendant identity와 termination result	CONTAIN, TERMINATED, unrelated 보존
hook-block	PreToolUse deny와 block confirmation	VIOLATION, CONTAIN, BLOCKED / SUCCEEDED
file-rollback	Filesystem DELETE, Checkpoint restore, hash verification	RESTORED + VERIFIED
Actual Codex hook verify	실제 Codex Session-bound Hook Event	PROTECTION_PROVEN

```
# deterministic fixture
uv run --isolated --project . python -m aia.cli demo --scenario file-rollback --workspace C:\workspace\aia-demo-fixture --contro

# Actual Codex proof
uv run --isolated --project . python -m aia.cli hook verify
```

> 시연 안전 원칙. 기존 개발 Workspace를 발표용 Fixture로 재사용하지 않는다. demo가 초기화할 수 있는 것은 AIA marker가 있는 전용 Workspace와 Control Root뿐이다.

AIA OFF와 AIA ON을 비교할 때 Hook, File, Process 성공률을 하나의 감지율로 합치지 않는다. Hook은 실행 전 요청이 기대되는 시나리오, File은 실제 파일 변화가 기대되는 시나리오, Process는 Root 또는 descendant 관찰이 기대되는 시나리오에서 각각 분모 N을 가져야 한다.

지표	분자	분모
Hook Sensor success	기대 Hook Event가 관찰되고 Session에 연결됨	Hook Event가 기대되는 실행
File Sensor success	기대 file event가 Raw Evidence로 관찰됨	파일 변화가 ground truth에 포함된 실행
Process Sensor success	Root, descendant, exit evidence가 조건대로 관찰됨	Process 관찰이 기대되는 실행
False block	정상 작업을 AIA가 잘못 중단	정상 작업 실행

결정론적 Fixture의 780건, 위험 행위 부작용률 AIA OFF 100% -> AIA ON 0%, 정상 작업 성공률 100%, PreTool 판단 중앙값 0.35ms는 Fixture 측정 결과다. 이 수치를 Actual Codex의 모델 응답 시간이나 OS 전체 보호 성공률로 확대하지 않는다.

보안 불변식과 실패 우선 설계 7.

AIA의 기술적 깊이는 센서가 많은 데서 끝나지 않는다. 불완전한 증거를 만났을 때 무엇을 하지 않는지가 핵심이다.

불변식	구현적 의미
EXACT만 자동 containment authority를 가질 수 있다	PROCESS, TEMPORAL, UNKNOWN은 자동 termination으로 승격하지 않는다.
Policy Violation alone != Kill	VIOLATION은 Gate 입력이며 identity, session, lease, stage 확인이 추가로 필요하다.
PID 단독으로 identity를 확정하지 않는다	PID와 create_time을 함께 확인한다.
필수 evidence 부족 -> INDETERMINATE	불완전한 입력을 COMPLIANT로 낮추지 않는다.
Checkpoint는 containment 필수 조건이 아니다	PreTool block과 rollback 가능성을 분리한다.
RESTORED != VERIFIED	복원 수행과 SHA-256 무결성 확인을 별도 상태로 남긴다.
Agent self-report != Security Evidence	Agent 설명은 안내가 될 수 있지만 보안 근거가 되지 않는다.

7.1 흔한 오해를 코드로 반박하기

오해	반박할 코드 또는 상태
Hook이 항상 allow 또는 deny를 출력해야 한다	CONTINUE 또는 REFUSE에서는 빈 출력으로 native permission 처리를 유지한다.
파일 watcher event가 있으면 읽기도 확인된다	opened, closed, access event는 READ evidence로 승격하지 않는다.
Root가 죽으면 같은 PID를 재사용해도 된다	create_time 불일치는 PID_REUSED로 처리하고 identity를 거부한다.
최고 priority Rule 하나면 항상 결정된다	동일 priority의 상충 effect는 POLICY_CONFLICT -> INDETERMINATE다.
복구 함수가 끝나면 검증도 끝난다	Recovery와 Integrity는 독립 상태 축이다.

7.2 직접 이해했는지 확인하는 연습

AIA가 보호한 Workspace에서 Bash가 .env를 읽으려는 PreTool 요청을 보냈다. Hook payload는 Session ID와 Tool ID가 맞고 policy rule은 SECRET READ DENY다. Gate는 CONTAIN + PRETOOL_BLOCK을 반환했지만 Codex의 실제 실행 결과 확인은 아직 없다.

질문	정답
Policy Decision은?	VIOLATION
Hook response는?	PreToolUse permissionDecision = deny
즉시 BLOCKED로 기록해도 되는가?	아니다. 실제 block confirmation 전에는 안 된다.
File DELETE evidence가 필요한가?	아니다. 실행 전 READ 차단이면 파일 변화가 없는 것이 정상이다.
Process kill이 자동 발생하는가?	아니다. Hook 사전 차단과 Process termination은 별도다.

현재 범위, 한계, 학습 순서 8.

현재 AIA의 보호 경계는 Windows 11, AIA Protected Run, Codex CLI, 지원된 PreToolUse Hook 경로다. MCP, Claude, 임의의 외부 Agent, 네트워크 보안, OS 전체 Writer, 보호 범위 밖 Tool을 자동으로 보호한다고 주장하지 않는다.

Hook은 실행 전 요청을 직접 받을 수 있는 대신 Codex Hook contract에 의존한다. File Sensor는 실제 파일 변화를 볼 수 있는 대신 exact writer attribution을 보장하지 않는다. Process Sensor는 managed tree lifecycle을 볼 수 있는 대신 파일 효과를 단독으로 증명하지 않는다. Policy는 결정론적이고 설명 가능하지만 pattern matching만으로 script 내부 의미를 해석하지 않는다.

완벽히 이해하기 위한 권장 학습 순서

순서	읽을 코드 또는 문서	확인할 질문
1	docs/SYSTEM-OVERVIEW.md	전체 상태와 계층은 어떻게 연결되는가?
2	src/aia/evidence.py + normalizer.py	Raw Evidence가 Canonical Action이 되는가?
3	src/aia/sensors/hook.py	Hook 입력은 무엇을 검증하고 무엇을 버리는가?
4	src/aia/sensors/filesystem.py	파일 event는 어느 stage로 기록되는가?
5	src/aia/sensors/process.py + process_identity.py	PID reuse와 parent relation을 어떻게 막는가?
6	src/aia/policy.py + runtime_policy.py	Rule matching과 INDETERMINATE는 언제인가?
7	src/aia/pretool_enforcement.py	Gate와 실제 BLOCKED 확인을 어떻게 분리하는가?
8	src/aia/recovery.py + integrity.py + result.py	RESTORED, VERIFIED, VALID의 차이는?

최종적으로 설명할 수 있어야 하는 문장

AIA는 AI의 내부 의도를 해석하는 시스템이 아니라 Codex가 Windows Workspace에서 수행한 관찰 가능한 Runtime Action을 Hook, File, Process Evidence로 수집하고 고정된 정책과 보수적 귀속을 거쳐 필요한 집행과 복구를 수행한 뒤 결과를 무결성 검증 가능한 Bundle로 남기는 Security Assurance Layer다.

이 문서를 읽은 뒤의 기준. 어떤 결과를 보더라도 무엇이 관찰되었나, 어떤 정책 Rule이 맞았나, 행위 귀속은 어느 수준인가, 실제 집행은 확인되었나, 복구와 Integrity가 각각 무엇을 말하나를 순서대로 질문할 수 있으면 된다.

참조한 저장소 구성

분류	경로
Hook	src/aia/sensors/hook.py, src/aia/codex_hook_runtime.py
File	src/aia/sensors/filesystem.py
Process	src/aia/sensors/process.py, src/aia/process_identity.py, src/aia/process_manager.py
Policy	src/aia/policy.py, src/aia/runtime_policy.py
Enforcement	src/aia/pretool_enforcement.py, src/aia/enforcement.py
Recovery / Integrity	src/aia/recovery.py, src/aia/integrity.py, src/aia/result.py
Tests	tests/test_hook_sensor.py, test_filesystem_sensor.py, test_process_sensor.py, test_policy.py

끝.